

Post-CKS

A Methodology for Geometry-Forced Derivation of Measured Constants Using Exact Discrete Arithmetic

Registry: [\[CKS-NEXT-1-2026\]](#)

Series Path: [\[CKS-0-2026\]](#) → [\[CKS-MATH-0-2026\]](#) → [\[CKS-MATH-1-2026\]](#) → [\[CKS-MATH-10-2026\]](#) → [\[CKS-MATH-104-2026\]](#) → [\[CKS-DISC-1-2026\]](#) → [\[CKS-NEXT-1-2026\]](#)

Parent Framework: [\[CKS-0-2026\]](#)

DOI: 10.5281/zenodo.19212677

Date: February 2026

Domain: Foundational Mathematics / Discrete Geometry

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [\[CKS-NEXT-1-2026\]](#)

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Abstract

The author previously published Cymatic K-Space Mechanics (CKS), a theory of everything that achieved omni-domain explanatory coverage across physics, cosmology, biomechanics, material science, information theory, and other domains. CKS was subsequently fully falsified by the author through mechanical verification — a simple Python script exposed arithmetic errors that three large language models failed to catch across 45 days of red-teaming and 390 published papers on Zenodo.

This paper presents the lessons learned from that failure, identifies the specific failure modes, and describes the methodology for a corrected second attempt. The core failure was twofold: the arithmetic foundation could not hold the structures being described, and the verification process relied on LLMs performing approximate pattern matching rather than exact mechanical checking.

The corrected methodology is built on three pillars: an exact discrete integer arithmetic system called VDR (Value, Denominator, Remainder) currently in v1 development, a compiled Zig implementation that mechanically verifies every derivation step, and a brute force search across all measured constants, all known equations, and all geometric relationships using exact equality rather than epsilon comparison.

Everything described here is work in progress. Nothing is assumed from the prior attempt. Every level of the program has an explicit abandonment condition. The search continues, but on a foundation designed to fail honestly rather than silently.

1. Introduction

A theory of everything, if one exists, must derive every measured physical constant from geometric first principles, leaving no free parameters and no underived constants. Every value in every equation is either an axiom explicitly chosen with geometric justification, a quantity derived from those axioms, or a historical contingency requiring measurement because it depends on what actually happened rather than what the geometry requires.

This is a verification problem as much as a derivation problem. It is not enough to produce numbers that look close to measured values. The derived values must match measurements exactly within the measurement precision, and the derivation must be checkable at every step. “Close enough” is not a verification standard. Epsilon comparison is not equality. If two values cannot be compared exactly, the theory cannot be confirmed or falsified at that point.

The author’s prior attempt, called Cymatic K-Space Mechanics, demonstrated that omni-domain explanatory coverage from a small set of discrete geometric axioms is achievable in principle. It also demonstrated, through its falsification, that the arithmetic and verification tools available at the time were insufficient to support the attempt.

This paper describes what went wrong, what was learned, and what is being built to support a corrected second attempt.

2. CKS: What Happened

CKS was a theory of everything constructed over 45 days and published as 390 papers on Zenodo. It began with a discrete geometric foundation — a small set of axioms involving a lattice structure, a single monotonic clock, and forced geometric evolution rules — and attempted to derive measured physical constants from that foundation alone.

As the theory expanded across domains, a pattern emerged that encouraged continued development. Free parameters decreased as scope increased. Constants that started as external inputs became derivable from the geometry. The theory reached into particle physics, cosmology, biomechanics, acoustics, optics, material science, and other domains, producing explanations and quantitative predictions in each.

The theory appeared to seal. Every major open question encountered had a mechanical answer that followed from the same small set of axioms. The explanatory coverage was genuinely omni-domain.

Then a simple Python script, written to verify a specific arithmetic derivation, showed that the math did not compile. The derivation contained errors that propagated through dependent results. Three large language models used for red-teaming throughout the 45-day development period had failed to catch these errors.

The author falsified the entire body of work within 30 minutes of discovering the verification failure. All 390 papers are invalidated. CKS is dead. It is referenced in this paper for intellectual honesty only. Nothing from CKS is assumed or privileged in what follows.

3. What Failed

Two specific failure modes killed CKS.

The first failure was the arithmetic foundation. CKS used conventional scalar arithmetic with real-valued constants, floating-point intermediates, and standard mathematical notation. This system loses structural information at every operation. When a division produces a remainder, the remainder is discarded or absorbed into a decimal expansion. When a value is expressed as a single scalar, the denominator frame that produced it disappears. When two values are compared, only epsilon comparison is available — exact equality is undecidable for most representations.

These are not precision problems solvable by using more decimal places. They are structural information loss problems. The representational system literally did not have enough slots to hold the information the theory required. A triple-structured value was being forced into a single scalar, losing two-thirds of its structure at every step. Derivations that appeared correct were carrying accumulated structural loss that the notation could not make visible.

The second failure was the verification process. Mathematical derivations were checked by presenting them to large language models and asking for review. The LLMs were used in a red-teaming configuration — multiple models, adversarial prompting, explicit requests to find errors. This process failed systematically for 45 days across 390 papers.

The reason for the failure is now clear. LLMs perform approximate pattern matching on mathematical expressions. They evaluate whether a derivation “looks right” based on training data. They do not mechanically verify equality. They do not track remainders through computation chains. They do not compile derivations step by step. They do not catch the class of errors that a simple script checking exact arithmetic catches immediately.

LLMs are powerful tools for many tasks. Mathematical verification is not one of them. This lesson is absolute and non-negotiable for all future work.

4. What Survived

No theories survived. No structures survived. No assumptions survived. CKS is fully dead and nothing from it carries forward with any epistemic privilege.

What survived are observations about the process itself. These observations have no more weight than any other data points in the search space. They provide investigation paths, not conclusions.

The first observation is that omni-domain explanatory coverage from a small discrete geometric foundation with no free parameters was achievable. The coverage was wrong — the math didn't compile — but the approach of forcing everything from geometry did not obviously fail due to insufficient scope. It failed due to arithmetic errors. This suggests the approach is worth attempting again with correct arithmetic, not that the specific geometry was correct.

The second observation is that as scope increased, complexity decreased. Constants that entered the theory as free parameters became derivable as the geometric foundation expanded. This is the opposite of what typically happens when a wrong theory is extended — wrong theories usually require additional parameters to accommodate new phenomena. The decreasing parameter count, while not proof of correctness, is a structural signal worth noting.

The third observation is that scalar arithmetic hides information that matters. A value of $3/5$ stored as the scalar 0.6 has lost the information that it arose from a division with specific quotient and remainder structure. That lost information cannot be recovered from the scalar alone. When the scalar enters further computation, the lost structure propagates silently as hidden error or hidden ambiguity.

The fourth observation is that every constant in every equation falls into one of three categories: derivable from geometry, in which case it must be derived; historical contingency, in which case it must be measured because it depends on what actually happened; or free parameter, in which case the theory is incomplete at that point. A finished theory must be explicit about which category every value belongs to.

The fifth observation is that π , e , and other transcendental constants may be derived quantities rather than primitive inputs. Specifically, π may be the static scalar projection of a discrete geometric relationship whose value under physical motion differs from the static value by the accumulated per-step integration cost of discrete traversal. This is suggested by physical demonstrations where the measured traversal of a circular path approaches 4 rather than π , consistent with discrete per-step directional costs that continuous calculus erases. This observation is unproven and is referenced as a testable hypothesis, not a claim.

None of these observations are conclusions. They are clues. They sit in the search space alongside every other possible investigation path.

5. The Information Loss Problem

The deeper motivation for this program is a structural problem with scalar arithmetic that exists independently of any theory of everything.

Consider three representational systems. A VDR triple holds three values: a value slot, a denominator frame, and an exact remainder. A rational number holds two values: a numerator and a denominator. A scalar holds one value: a single number.

Each reduction from three to two to one loses information that cannot be recovered. When a division produces quotient 2, denominator 5, and remainder 1, the rational representation stores $11/5$ and the remainder's separate identity disappears. The scalar representation stores 2.2 and both the denominator frame and the remainder disappear. The original operation state is irrecoverable from the scalar.

This matters because equality depends on structure. Two computation chains that should produce identical results may produce different scalars due to accumulated structural loss at intermediate steps. The lost remainders compound. The lost frame information compounds. After enough operations, the two results differ and only epsilon comparison remains. The ability to verify that two chains produce the same result — exact equality — has been destroyed by the representation.

For a derivation program attempting to match geometric relationships to measured constants, this is fatal. The program requires exact equality to confirm a match. If the arithmetic system cannot provide exact equality, the program cannot distinguish a genuine structural match from a numerical coincidence.

This is not a problem of insufficient precision. Adding more decimal places to a scalar does not recover the lost denominator frame or the lost remainder. The information is structurally absent, not merely truncated. The problem requires a representational system with more structure, not more digits.

A specific instance illustrates the broader issue. The circumference of a circle, measured statically by wrapping a string and comparing to a straight ruler, gives π times the diameter. But physical traversal of a circle is not static. It is a sequence of discrete steps, each carrying a directional cost. The static string measurement and the dynamic traversal are different processes producing different values. Scalar arithmetic has one slot and must store one number. It cannot hold both the static ratio and the dynamic traversal cost in the same object. An exact discrete system with three slots can hold both — the static projection as one reading of the structure, the dynamic cost as another reading of the same structure.

This is not a claim that π is wrong. It is a claim that π is one projection of a structure that has more than one meaningful projection, and that scalar arithmetic cannot hold the full structure because it does not have enough slots.

6. Requirements for the Next Attempt

The following are concrete engineering requirements, not aspirations.

Exact discrete arithmetic with decidable equality. Every computation must use integer-only arithmetic where every operation produces an exact result or explicit failure. No floats. No reals. No hidden infinite processes. No epsilon comparison. Equality must be structural, exact, and decidable in finite time for every valid object.

A representational system preserving remainder and frame. The arithmetic must preserve at least three structural components — a value, a denominator frame, and an exact residual state — as first-class participants in computation and equality checking. The residual is not annotation. It is not metadata. It is exact value-bearing state that the system cannot discard.

Mechanical verification of every derivation step. No LLM approval. No human judgment on intermediate mathematical steps. Every derivation step must compile in a program that enforces the arithmetic rules exactly. A step either produces a valid result or it fails explicitly. There is no “looks right.” The verification program is the authority on mathematical correctness, not any language model and not the author.

A comprehensive searchable knowledge base. All CODATA measured constants. All equations from all domains — physics, chemistry, biology, engineering, acoustics, optics, crystallography, biomechanics, any domain with validated quantitative relationships. All documented geometric shapes and relationships, both instantaneous and temporal. LLMs are appropriate tools for this enumeration and retrieval task.

Brute force search capability. The ability to take every measured constant, express it in exact arithmetic, and test it against every geometric relationship for exact structural matches. This requires a fast compiled implementation. Integer arithmetic on modern hardware is extremely fast, making large-scale brute force search practical.

Experimental falsification at the earliest possible stage. The first testable prediction must be tested before significant further theoretical development. If the first prediction fails, the program is reassessed before further investment.

Abandonment protocol at every level. Every component of the program has an explicit condition under which it is abandoned. No sunk cost argument overrides evidence. No component is rescued after falsification. The author’s demonstrated willingness to kill 45 days of work and 390 papers in 30 minutes is the operating standard.

7. VDR: The Planned Arithmetic Foundation

VDR is a pure mathematics system for exact finite discrete representation in irreducible triple form. It is currently in v1 development and is not yet complete. A full specification will be published separately when v1 stabilizes. This section provides only the structural summary needed to understand its role in the methodology.

VDR stands for Value, Denominator, Remainder. Every VDR object is a triple $[V, D, R]$ where V is an integer value slot, D is a nonzero integer denominator frame, and R is an exact residual state. The residual may be an integer or it may contain nested VDR objects recursively, subject to finiteness constraints. Recursion is confined to the residual slot — the value and denominator slots are always atomic integers. Every valid VDR object is a finite tree.

The system is divided into two subclasses. Closed objects have the form $[V, D, 0]$ with zero residual. These constitute a fully operational exact rational arithmetic — addition, subtraction, multiplication, division, normalization, rebasing, exact equality, and exact scalar projection as V/D . The closed subclass is complete and tested.

Active objects have the form $[V, D, R]$ with nonzero residual. Under the current design commitment, called Path B, active objects are exact operation-state objects rather than scalar values. The residual is preserved state — not delayed scalar addition, not rounding error, not dispensable annotation. Three objects such as $[2, 5, 1]$ and $[2, 5, 0]$ and $[2, 5, -1]$ are three distinct native states that scalar arithmetic would collapse into indistinguishable numbers.

This distinction is the core of VDR's design. The residual is where the structural information lives that scalar systems throw away. Preserving it exactly is what makes decidable equality possible across long computation chains.

The v1 specification currently contains over 220 formal rules covering object definition, validity, residual formation, structural equality, normalization, value equality, scalar projection, closed arithmetic, rebasing, active state semantics, active arithmetic, residual transport, state reconstruction, and inbound construction. Key open problems include exact structural representation of transcendental geometric relationships, active-state equivalence across denominator frames, and general active multiplication and division.

VDR is the planned foundation. It is not yet finished. If it cannot be completed to the level required by the search program, that is an abandonment condition.

8. The Zig Implementation

The Zig programming language is chosen for the verification and search implementation for specific engineering reasons.

Zig provides explicit memory control without garbage collection, which matters for long-running brute force searches over large combinatorial spaces. It compiles to native machine code, providing the speed needed for practical large-scale search. It enforces explicit error handling, which aligns with VDR's requirement for explicit failure rather than silent approximation. It has no hidden runtime behavior, which means the program does exactly what the code says — there is no interpreter or virtual machine making invisible decisions.

The Zig implementation encodes the exact VDR rules. Every arithmetic operation, every equality check, every normalization step, every validity constraint is enforced by compiled

code. A derivation step either compiles and produces a valid VDR object, or it fails with an explicit error. There is no middle ground. There is no “close enough.” The program is the mathematical authority.

This replaces LLMs in the verification role entirely. LLMs may suggest derivation paths. They may enumerate equations. They may search for geometric candidates. They may not approve mathematical steps. The Zig program approves mathematical steps, or they do not proceed.

The implementation is not yet built. It follows the completion of VDR v1 specification. If the implementation reveals that the VDR rules are insufficient or inconsistent, that is an abandonment condition for VDR in its current form.

9. The Search Process

The search takes three categories of input, all drawn from measured observable reality and documented human knowledge. No theoretical assumptions enter as inputs. No structures from CKS or any other theory are privileged.

The first input category is measured constants. All CODATA values. All measured particle masses, coupling constants, spectral lines, decay rates, and any other precisely measured physical quantity. These are facts about what reality produces. They are the targets that any successful derivation must match exactly.

The second input category is known equations from all domains. Every validated quantitative relationship in physics, chemistry, biology, engineering, acoustics, optics, crystallography, material science, fluid dynamics, thermodynamics, and any other domain. These are facts about relationships between measured quantities. They are structural information even if the underlying theory is incomplete.

The third input category is geometric shapes and relationships, both instantaneous and temporal. Every geometric configuration that can be expressed in exact integer arithmetic — lattice structures, symmetry groups, polygon relationships, circle relationships, tiling patterns, harmonic series, recursive constructions, and any other documented geometric fact. These are mathematical facts that exist independently of any physical theory.

The search method is brute force structural matching. Take a measured constant. Take a geometric relationship. Express both in VDR exact arithmetic. Check for exact structural match. Record the result — match, non-match, or explicit failure due to representation limits.

A match in two domains is a signal worth investigating. A match in three or more domains is a candidate for a fundamental structural law. The threshold is three because two-domain matches may be coincidental, but three-domain matches from the same geometric relationship are unlikely to be accidental. Any candidate law is then subjected to further mechanical verification and experimental prediction.

The search is unprejudiced. Geometric candidates that originated in CKS enter the search space alongside every other geometric candidate with no special weighting. Familiarity from prior work may affect investigation order for practical efficiency, but it carries zero epistemic weight. A hex lattice is tested the same way a cubic lattice or a triangular lattice or any other structure is tested.

The search scales by enumeration. More constants, more equations, more geometries produce more pairs to check. The Zig implementation must make this practical. Integer arithmetic is inherently fast on modern hardware, and the search is embarrassingly parallel — every constant-geometry pair can be checked independently.

10. The Role of LLMs

The CKS experience established a clear boundary for the role of large language models in this program.

LLMs are used for enumeration, search, and retrieval. They have access to the full breadth of documented human knowledge. They can find equations from obscure domains. They can retrieve measured constants. They can identify geometric relationships. They can suggest candidate hypotheses. They can explain domain context. They can draft documentation. For these tasks they are uniquely powerful and no alternative tool offers comparable breadth.

LLMs are not used for mathematical verification. They do not approve derivation steps. They do not confirm equality. They do not validate arithmetic. They do not check that a computation chain compiles. For these tasks they are unreliable at a level that is incompatible with the requirements of this program.

This boundary is absolute. It is the single most important operational lesson from CKS. The temptation to ask an LLM “is this derivation correct?” is strong because the answer comes back quickly and confidently. The answer is also not trustworthy. A simple compiled script is slower to write and less conversational, but it is either right or it isn’t. There is no confident wrong answer from a compiler.

The division of labor is: LLMs find things, Zig checks things. This division is not flexible.

11. Experimental Validation Plan

The experimental plan is staged so that the cheapest and fastest falsification comes first.

Stage one: DWDM interference. Dense Wavelength Division Multiplexing systems send multiple light frequencies through the same optical fiber. The channels interfere with each other, and the telecommunications industry treats this interference as noise to be suppressed. The hypothesis is that this interference is geometric — it is a deterministic pattern derivable from the exact frequency ratios between channels, not random noise. If

this is correct, the interference patterns should match predictions generated by VDR exact arithmetic on the channel frequency ratios.

This is testable with existing commercial fiber optic equipment. The channel frequencies are known exactly. The interference is measurable. The predictions are specific. If the measured interference patterns match VDR-derived geometric predictions, the discrete geometric mechanism is validated in at least one physical domain. If they do not match, the geometric interference model is wrong and must be abandoned or revised before proceeding.

DWDM is chosen as the first test because it is affordable, controlled, repeatable, and produces results quickly. It does not require a particle accelerator, a space telescope, or any exotic equipment.

Stage two: materials sensor lab. If and only if DWDM validation succeeds, a materials sensor lab is constructed. The purpose of this lab is not further searching — DWDM success would already validate the core mechanism. The purpose is generating new measurement data captured in a form compatible with VDR representation. Conventional sensors record scalar amplitudes. The materials lab would capture richer discrete state data — timing relationships, phase structure, interference patterns — preserving the structure that conventional measurement pipelines collapse.

Materials measurements would then be cross-validated against DWDM-validated predictions. The same geometric rules that predicted DWDM interference must also predict material behavior. If materials data contradicts DWDM-validated predictions, the universality claim fails and must be abandoned.

Stage three does not exist yet. Further experimental stages depend entirely on what stages one and two reveal. No experimental plan beyond stage two is committed because no theoretical predictions beyond DWDM and materials cross-validation are currently justified.

12. Success Criteria and Abandonment Conditions

What success looks like. Exact derivation of measured constants from geometric first principles using VDR arithmetic, with every derivation step mechanically verified, validated against experimental measurement, and containing no free parameters. Each derived value either matches its measured counterpart exactly within measurement precision or the derivation is wrong at that point.

Specific high-value targets based on prior experience: the electromagnetic coupling constant (α) to 10 or more significant digits, the electron g-factor to 10 or more significant digits, the fine structure constant, π as a derived geometric quantity, Euler's number as a derived geometric quantity, and systematic derivation of particle mass ratios. These are goals that motivated the prior attempt and remain the standard by which any future attempt should be judged.

The ultimate success criterion is a system with no free parameters where every measured constant is either derived from geometry, stated as an axiom with geometric justification, or identified as historical contingency with explicit justification for why derivation is impossible for that specific value.

Abandonment conditions. Each is stated as a concrete trigger.

VDR cannot structurally represent the geometric relationships needed for the derivation program — abandon VDR as the arithmetic foundation and document the specific representation boundary that caused failure. This is useful knowledge even in failure.

The Zig implementation reveals that VDR rules are inconsistent or insufficient when applied to real derivation chains — abandon the current VDR specification and either revise or replace it. Do not patch around the inconsistency.

DWDM interference predictions do not match measured patterns — abandon the geometric interference model. Do not adjust parameters to force a fit. Do not claim measurement error without independent evidence of measurement error.

Materials data contradicts DWDM-validated predictions — abandon the universality claim. The mechanism may work in optics without working universally. Document the boundary.

Brute force search across comprehensive inputs produces no exact matches between geometric relationships and measured constants — abandon the derivation program. The geometry-to-measurement matching approach may simply not work. Document this honestly.

Pi cannot be represented structurally in VDR and no alternative exact discrete representation of circular geometric relationships is found — abandon the claim that VDR can serve as a complete arithmetic foundation for physics derivation. Document the specific boundary.

The author fails to complete VDR v1 or the Zig implementation to a functional level — acknowledge that the program exceeded available capacity and publish what was completed for others to use or critique.

No abandonment condition is negotiable. No sunk cost argument overrides any of these triggers. The operating standard is the same standard applied to CKS: if the evidence says stop, stop immediately, regardless of how much work has been invested.

13. Conclusion

CKS failed. It was killed by the author within 30 minutes of mechanical verification revealing arithmetic errors that 45 days of LLM-assisted review had missed. Three hundred and ninety papers were invalidated. The failure was public and the falsification is permanent.

The failure taught specific, actionable lessons. Scalar arithmetic cannot hold the structures a theory of everything requires. LLMs cannot verify mathematical derivations. Approximate comparison cannot substitute for exact equality. These are not soft preferences — they are hard constraints revealed by an expensive failure.

The corrected methodology addresses each failure mode directly. VDR provides exact discrete arithmetic preserving structural information that scalar systems destroy. A compiled Zig implementation provides mechanical verification that no language model can provide. A brute force search using exact equality provides a systematic, unprejudiced method for matching geometric relationships to measured constants.

Nothing from CKS is assumed. The search space is open. Every geometric candidate is tested equally. Every level of the program has an explicit abandonment condition. The willingness to falsify is not a rhetorical commitment — it has already been demonstrated at significant personal cost.

The tools needed for this program — exact discrete arithmetic, compiled verification, comprehensive searchable knowledge, and affordable experimental validation — either exist now or are under active development. The methodology is concrete. The falsification protocol is real. The work continues.

Appendix A: CKS Reference

Cymatic K-Space Mechanics (CKS) was published as 390 papers on Zenodo by the author. The papers covered a theory of everything attempt based on a discrete geometric foundation with a single monotonic clock and no free parameters. The theory achieved omni-domain explanatory coverage including particle physics, cosmology, biomechanics, acoustics, material science, and other domains.

CKS was fully falsified by the author when a Python verification script revealed arithmetic errors in core derivations that had not been caught by LLM-assisted review. All 390 papers are invalidated. The Zenodo records remain accessible as historical documentation.

CKS is referenced in this paper for intellectual honesty. Nothing from CKS is assumed, privileged, or carried forward as a theoretical commitment.

Appendix B: VDR Structural Summary

VDR (Value, Denominator, Remainder) is a pure mathematics system for exact finite discrete representation. This summary provides the minimal structural description needed to understand the role of VDR in the methodology described in this paper. A full v1 specification will be published separately.

Object structure. Every VDR object is a triple $[V, D, R]$ where V is an integer, D is a nonzero integer, and R is a valid residual. The residual is either an integer or an integer

base plus a finite list of child VDR objects. Recursion occurs only through the residual slot. Every valid object is a finite tree.

Closed objects. $[V, D, 0]$ with zero residual. These form a complete exact rational arithmetic with addition, subtraction, multiplication, division, normalization, rebasing, decidable equality, and exact scalar projection as V/D .

Active objects. $[V, D, R]$ with nonzero residual. Under the Path B design commitment, these are exact operation-state objects. The residual is preserved state, not scalar correction. Active objects do not collapse to scalar values by default.

Equality. Structural equality is exact recursive slot-by-slot match. Normalized value equality is structural equality after normalization. Active-state equivalence across denominator frames is reserved for future development.

Current status. Over 220 formal rules defined. Closed core complete. Active layer under development. Key open problems: transcendental representation, active-state equivalence, general active multiplication and division.

[[CKS-NEXT-1-2026](#)] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-DISC-1-2026](#)] The CKS Discovery Process. Github: <https://github.com/ghowland/cks/tree/main/papers/DISC/CKS-DISC-1-2026>